DOCUMENT 17

NGW Lab Manual

Signal pipeline, benchmark system v2, reference dataset, VLM architecture,

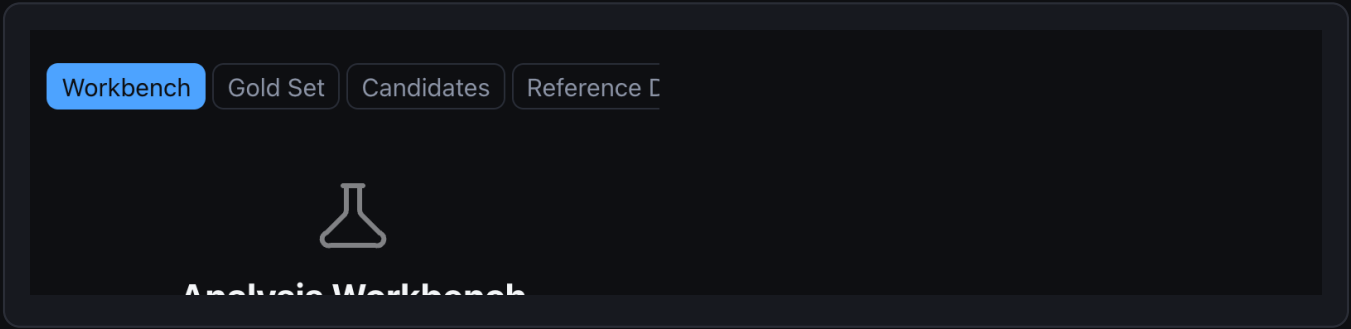
NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential

NGW

About This Manual

The NGW Lab is the quality-control and continuous-improvement backbone of the No Guesswork engine. Curators use it to record and analyse outcome signals from real sessions, build and maintain the benchmark test-case library, manage gold-set reference images, propose and track rule candidates, and run the full suite of validation scripts. This manual documents every surface, API endpoint, database table, and CLI tool in the Lab — drawn directly from the source.

7-Stage	19	28	v2
VLM analysis pipeline	Canonical blueprints	Reference patterns	Benchmark system



NGW Lab — four-tab layout: Workbench, Gold Sets, Candidates, Signals

ACCESS CONTROL

- Lab features are gated by NGW_DEV_EMAILS — a comma-separated allowlist in .env.
- All Lab API endpoints require a valid JWT whose email appears in NGW_DEV_EMAILS.
- Set NGW_DEV_MODE=1 to bypass auth locally. NEVER set this in production.
- Enable in UI: Settings Developer Tools toggle on enter your email.

Role	Auth requirement	Key capability
Curator	JWT + NGW_DEV_EMAILS listed	Gold sets, candidates, benchmarks, signals
Second reviewer	JWT + NGW_DEV_EMAILS listed	Approve high-risk rule candidates
CI system	X-CI-Secret header or dev JWT	POST /api/lab/benchmarks/ci-run
Unauthenticated	None	POST /api/lab/signals (session recording)

Section 1 — Signal System

Every user session that produces an outcome is recorded as exactly one signal in the session_signals table. Signals are the primary data source for calibration analysis, pattern performance metrics, and automated candidate generation. The recording call is synchronous — fired at the moment the user submits feedback.

Lab → Signals — hygiene card with live / seeded / internal / analytics-eligible counts

Signal Schema — session_signals

Column	Type	Description
id	TEXT PK	Auto UUID hex
pattern_id	TEXT	Pattern attempted — required
outcome	TEXT	nailed_it | close | failed | unknown
confidence_score	REAL	Engine confidence 0.0–1.0
input_method	TEXT	wizard | reference_photo | manual
subject_type	TEXT	woman | man | child | couple | group
environment	TEXT	studio | indoor | outdoor
mood	TEXT	beauty | cinematic | corporate | editorial | natural

shoot_mode_entered	INT	1 if user opened Shoot Mode for this pattern
steps_completed	INT	Shoot Mode steps finished before feedback
steps_total	INT	Total steps in the assigned sequence
deviation_count	INT	Test-vs-reference evaluation deviations
saved_setup	INT	1 if user saved this setup to My Kit
upgraded	INT	1 if session led to subscription upgrade
revenue_value	REAL	Monetization attribution amount (if upgraded)
signal_source	TEXT	live | seeded | internal | expert_review
include_in_learning	INT	Eligible for candidate auto-generation (default 1)
include_in_metrics	INT	Included in analytics dashboards (default 1)
include_in_conversion	INT	Included in CVR calculations (default 1)
include_in_cohorts	INT	Included in cohort analysis (default 1)
session_id	TEXT	Optional — user session identifier
user_id	TEXT	Optional — user identifier
created_at	REAL	Unix timestamp (unixepoch() default)

Signal Source Taxonomy

Source	Meaning	include_in_* default
live	Real user session — production outcome	All = 1 (analytics eligible)
seeded	Bootstrap data from seed_starter_dataset.py	All = 0 (excluded)

internal	Dev / curator sessions during testing	All = 0 (excluded)
expert_review	Curator-flagged signal with correction	All = 0 (manually promoted)

ANALYTICS ELIGIBILITY RULE

- Only signal_source=live rows are included in metrics, conversion, and cohort analysis.
- Seeded and internal rows occupy the same table but with all include_in_* = 0.
- Expert-reviewed signals can be selectively promoted by setting include_in_learning=1 with a corrected pattern_id.
- The hygiene endpoint shows a real-time count of analytics-eligible vs excluded rows.

Signal API — Complete Reference

POST /api/lab/signals — record a session signal (public — no auth required)

```
{
  "pattern_id":          "rembrandt",
  "outcome":             "nailed_it",   # nailed_it | close | failed | unknown
  "confidence_score":    0.84,
  "input_method":        "reference_photo",
  "subject_type":        "woman",
  "environment":         "studio",
  "mood":                "beauty",
  "shoot_mode_entered":  true,
  "steps_completed":     6,
  "steps_total":         6,
  "deviation_count":     0,
  "saved_setup":         true,
  "upgraded":            false,
  "signal_source":       "live"
}
# Response
{ "success": true, "signal_id": "abc123",
  "outcome": "nailed_it", "signal_source": "live" }
```

GET /api/lab/signals/summary — headline KPIs (auth required)

```
# Query params: ?days=30&source=live
{
  "total_sessions":   842,
  "success_rate":     0.74,
  "top_pattern":      "loop",
  "worst_pattern":    "broad",
  "conversion_rate":  0.12,
  "avg_confidence":   0.79,
  "revenue_total":    1840.00
}
```

GET /api/lab/signals/patterns — per-pattern aggregation

```
# Query params: ?days=30&source=live
# Returns array — one entry per pattern:
{
  "pattern_id":     "loop",
  "success_rate":   0.81,
  "avg_confidence": 0.83,
  "outcomes": { "nailed_it": 48, "close": 22, "failed": 14 },
  "sample_count":   84
}
```

GET /api/lab/signals/calibration — confidence vs outcome mismatch

```
# Query params: ?days=30&source=live
# overconfident = true when gap > 0.20
{
  "pattern_id":       "broad",
  "avg_confidence":   0.78,
  "measured_success": 0.55,
  "gap":              0.23,
  "overconfident":    true
}
```

GET /api/lab/signals/recent — latest N signals

```
# Query params: ?limit=50&pattern_id=loop&source=live
# Default source=live. Each item:
# signal_id, pattern_id, outcome, confidence_score,
# input_method, environment, created_at, session_id
```

GET /api/lab/signals/hygiene — source breakdown (dev auth required)

```
{
  "by_source": {
    "live":          1482,
    "seeded":         500,
    "internal":        23,
    "expert_review":    8
  },
  "analytics_eligible": 1482,
  "flag_overconfident_patterns": ["broad", "clamshell"]
}
```

POST /api/lab/signals/seed — inject 45 bootstrap rows (dev auth)

```
# Query params: ?force=true  (required if seeds already exist)
# Inserts 45 synthetic rows with signal_source=seeded
# All include_in_* flags = 0  (excluded from all analytics)
# Response: { "seeded_count": 45, "skipped": 0 }
```

How To: Read a Calibration Report

1. **Open Lab Signals**
   Navigate to the Signals tab in the Lab navigation.

2. **Select Calibration**
   Tap the Calibration sub-tab — shows confidence vs outcome per pattern.

3. **Find overconfident flag**
   overconfident: true means engine confidence is 20+ pp above measured success.

4 **Check sample_count**

Ignore patterns with sample_count < 20. Noise, not signal.

5 **Compare the gap**

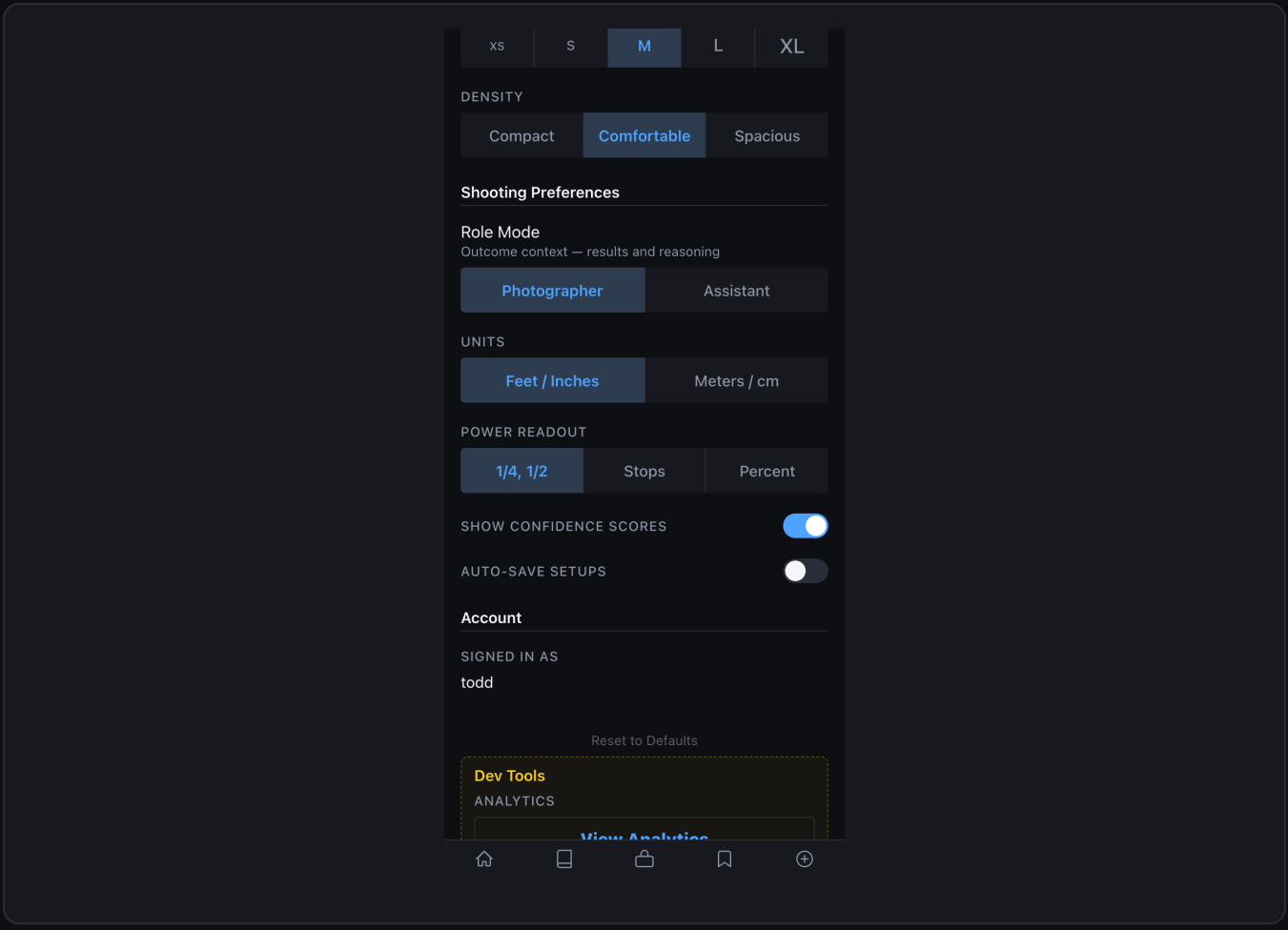
gap = avg_confidence – measured_success. Gap > 0.20 = actionable.

6 **Create a rule candidate**

If gap > 0.20 with 20+ samples: document in rule_candidates (Section 3).

Section 2 — Benchmark System v2

The benchmark system evaluates engine accuracy against a curated library of test cases. Each case carries ground-truth expected analysis and expected blueprint geometry. Results are scored across four weighted dimensions and compared against a pinned baseline. CI runs block deployment when any gate condition fails.



Lab Settings — benchmark thresholds and evaluation gate configuration

Scoring Model

Dimension	Weight	What it measures	Target

pattern_accuracy	40%	Correct pattern detected vs. expected	>= 0.80
blueprint_score	30%	Key/fill geometry matches expected positions	>= 0.75
fix_score	20%	Correct fixes proposed for failure-mode cases	>= 0.70
confidence_error	10%	abs(predicted_conf - measured_conf)	<= 0.15

overall_score = 0.40 * pattern_accuracy + 0.30 * blueprint_score + 0.20 * fix_score + 0.10 * (1.0 - confidence_error)

Database Tables

Table	Primary purpose
benchmark_cases	Test case library — image_path, expected_analysis, expected_blueprint, difficulty
benchmark_runs	Run records — timestamps, scores, status, regression_count
benchmark_results	Per-case result per run — predicted_pattern, all four scores, regression_flag
pattern_metrics	Rolled-up per-pattern scores with last_updated timestamp
gold_set_entries	Curator-verified reference images — source material for benchmark cases

Benchmark Case Schema — benchmark_cases

Column	Type	Description
id	TEXT PK	Auto UUID hex

pattern_id	TEXT	Pattern under test
image_path	TEXT	Path to reference photo (required)
difficulty	TEXT	easy | medium | hard (default: medium)
environment_tags	JSON	Array of environment descriptors
expected_analysis	JSON	Ground-truth: pattern_id, environment, subject_type, skin_tone, mood
expected_blueprint	JSON	Ground-truth: key/fill angle_deg, height, modifier family
expected_fixes	JSON	Array of expected fix recommendations (for failure-mode cases)
source_gold_set_id	TEXT	FK to gold_set_entries (optional — set via from-gold-set promotion)
notes	TEXT	Curator notes
created_by	TEXT	Curator email

POST /api/lab/benchmarks/cases — create a test case

```
{
  "pattern_id":  "loop",
  "image_path":  "data/reference_dataset/loop/gs_001/image.jpg",
  "difficulty":  "medium",
  "environment_tags": ["studio", "low_fill"],
  "expected_analysis": {
    "pattern_id":   "loop",
    "environment":  "studio",
    "subject_type": "woman",
    "skin_tone":    "fair",
    "mood":         "beauty"
  },
  "expected_blueprint": {
    "key":  { "angle_deg": 45, "height": 7.5, "modifier": "octa_90cm" },
    "fill": { "side": "camera_left", "type": "reflector" }
  },
  "expected_fixes": [],
  "notes":      "Classic loop. Clean catch-light. No ceiling bounce.",
  "created_by": "curator@org.com"
}
# Response: { "id": "case_uuid", "pattern_id": "loop", "difficulty": "medium" }
```

POST /api/lab/benchmarks/cases/from-gold-set/{id} — promote gold set entry to case

```
# No request body. Fields inferred from gold_set_entries.expected_analysis.
# Sets: source_gold_set_id, expected_analysis, image_path automatically.
# Response: New benchmark_case record with all inferred fields populated.
```

Running Benchmarks

POST /api/lab/benchmarks/run — trigger a manual run

```
{
  "run_type":   "manual",
  "trigger":    "curator@org.com",
  "case_limit":  null,
  "notes":      "Pre-deploy validation after loop threshold change."
}

# Response (summarised):
{
  "run_id":              "run_uuid",
  "status":              "completed",
  "total_cases":         28,
  "passed_cases":        25,
  "overall_score":       0.871,
  "pattern_accuracy":    0.893,
  "avg_blueprint_score": 0.842,
  "confidence_error":    0.118,
  "regression_count":    0,
  "blocked":             false
}
```

POST /api/lab/benchmarks/ci-run — CI/CD integration (X-CI-Secret or dev JWT)

```
# Automatically sets run_type=ci, trigger=ci-system
# Response includes exit_code field for shell integration:
{ "status": "completed", "exit_code": 0,
  "overall_score": 0.871, "message": "All gates passed" }

# exit_code = 1 when ANY of these conditions is true:
#   overall_score < 0.70
#   any single pattern passes < 75% of its cases
#   regression vs baseline > 5 percentage points
```

GET /api/lab/benchmarks/runs/{id}/results — per-case detail (sorted worst-first)

```
# Each result:
{
  "case_id":          "case_uuid",
  "pattern_id":       "broad",
  "difficulty":       "hard",
  "predicted_pattern":"loop",
  "pattern_correct":  false,
  "blueprint_score":  0.62,
  "fix_score":        0.50,
  "confidence_error": 0.24,
  "final_score":      0.577,
  "regression_flag":  true,
  "error_msg":        "Pattern mismatch: expected broad, got loop"
}
```

GET /api/lab/benchmarks/baseline + POST to promote

```
# GET — check current baseline
{ "has_baseline": true, "baseline": {
    "run_id": "run_uuid",
    "overall_score": 0.871,
    "pattern_accuracy": 0.893,
    "set_at": 1742000000 } }

# POST — promote latest completed run to active baseline
# Body: {}  (no parameters)
# Only promote when regression_count = 0 and overall_score >= previous baseline.
```

CI GATE RULES (NEVER BYPASS)

- overall_score < 0.70 exit_code 1, deploy blocked.
- Any pattern: pass rate < 75% exit_code 1, deploy blocked.
- Regression vs baseline > 5pp exit_code 1, deploy blocked.
- Never promote a baseline with regression_count > 0.

How To: Add a Benchmark Case

1. **Choose reference image**
   Clean, unedited photo with clear lighting geometry. Min 1024px wide.

2. **Confirm ground truth**
   Note actual setup: key angle, height, modifier, fill side, ratio.

3. **POST the case**
   POST /api/lab/benchmarks/cases with all expected_* fields populated.

4. **Set difficulty honestly**
   easy = textbook; medium = real-world; hard = ambiguous or edge-case.

5. **Or promote from gold set**
   POST /api/lab/benchmarks/cases/from-gold-set/{id} auto-infers fields.

6 **Run targeted eval**

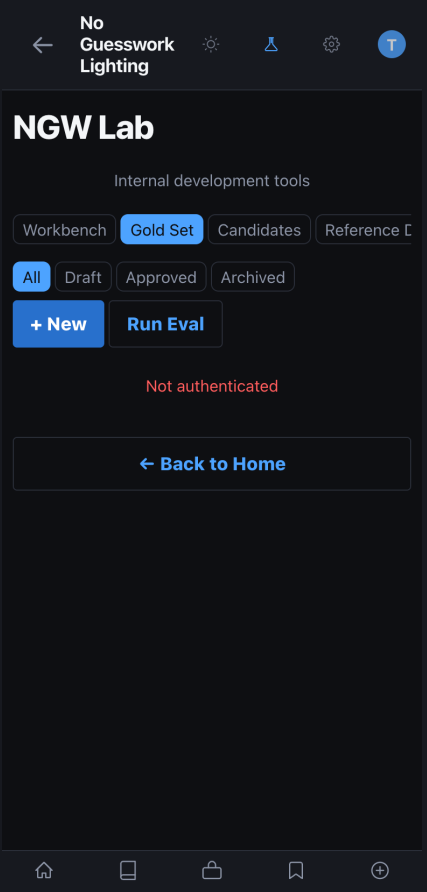
POST /api/lab/benchmarks/run with case_limit=1 to verify it scores correctly.

7 **Check pattern_correct**

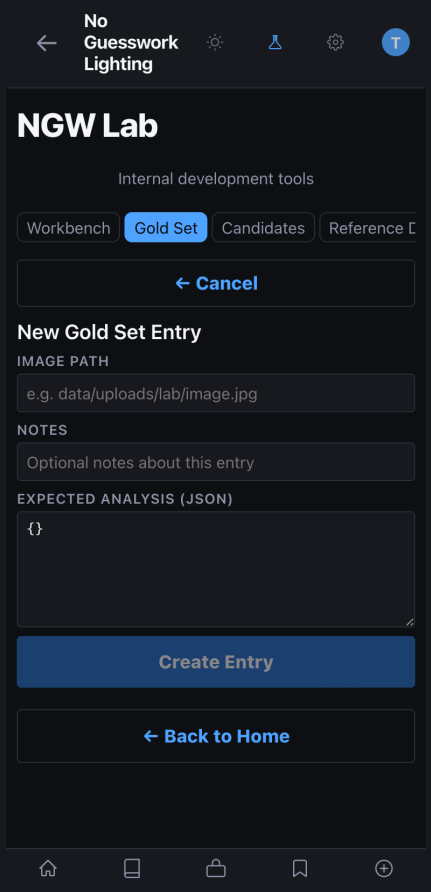
GET /api/lab/benchmarks/runs/{id}/results — confirm pattern_correct: true.

Section 3 — Gold Sets & Rule Candidates

Gold set entries are curator-verified reference images with ground-truth metadata. They feed benchmark cases via the from-gold-set promotion API and supply evidence for rule candidates. Rule candidates are structured improvement proposals derived from calibration gaps, benchmark failures, or direct curator observation.



Gold Set listing — images with pattern labels, status, and linked benchmark case count



New Gold Set form — image path, pattern, expected_analysis JSON, and curator notes

Gold Set Entry Schema — gold_set_entries

Column	Type	Description
id	TEXT PK	Auto UUID hex
image_path	TEXT	Path to the verified photograph (required)
expected_analysis	JSON	Ground-truth object (see field list below)
notes	TEXT	Curator observation notes
status	TEXT	draft | approved | rejected

created_by	TEXT	Curator email
created_at	REAL	Unix timestamp
updated_at	REAL	Unix timestamp (updated on status change)

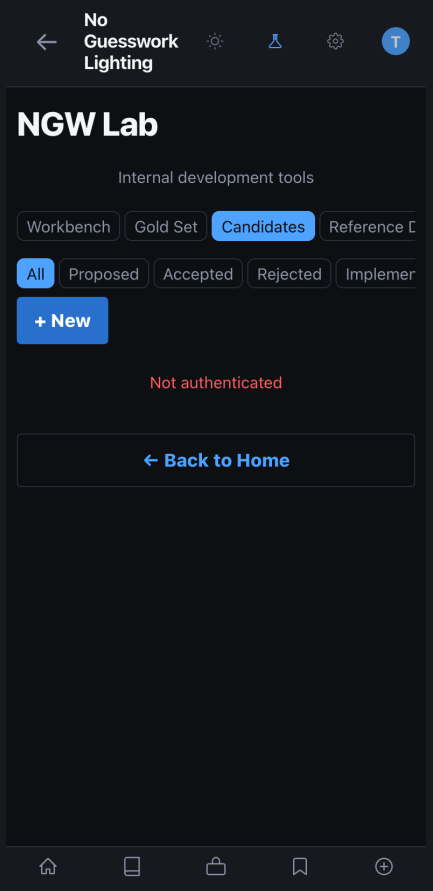
expected_analysis — all supported fields

```
{
  "expected_pattern":  "rembrandt",
  "lighting_family":   "directional",
  "pattern_id":        "rembrandt",
  "key_light":   { "position": "45deg", "modifier": "beauty_dish", "power": 1.0 },
  "fill_light":  { "position": "camera_left", "modifier": "reflector", "power": 0.…
  "rim_light":   { "position": "rear_right", "modifier": "strip_box", "power": 0.5…
  "environment":  "studio",
  "subject_type": "woman",
  "skin_tone":    "fair",
  "mood":         "editorial"
}
```

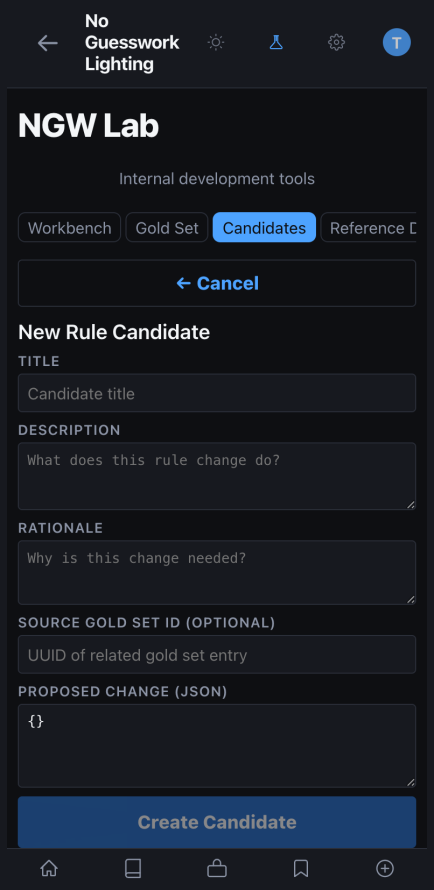
Rule Candidate Schema — rule_candidates

Column	Type	Description
id	TEXT PK	Auto UUID hex
title	TEXT	Concise rule description (required)
description	TEXT	Full rule specification (required)
rationale	TEXT	Evidence: calibration data, benchmark case IDs, gold set IDs
source_gold_set_id	TEXT	Optional FK to gold_set_entries

proposed_change	TEXT	Concrete JSON delta or patch specification
status	TEXT	proposed | under_review | approved | deployed
created_by	TEXT	Contributor email
created_at	REAL	Unix timestamp



Candidates queue — proposed, under_review, approved, and deployed entries with status badges



New Candidate form — title, description, rationale, proposed_change JSON, and linked gold set

Candidate Lifecycle

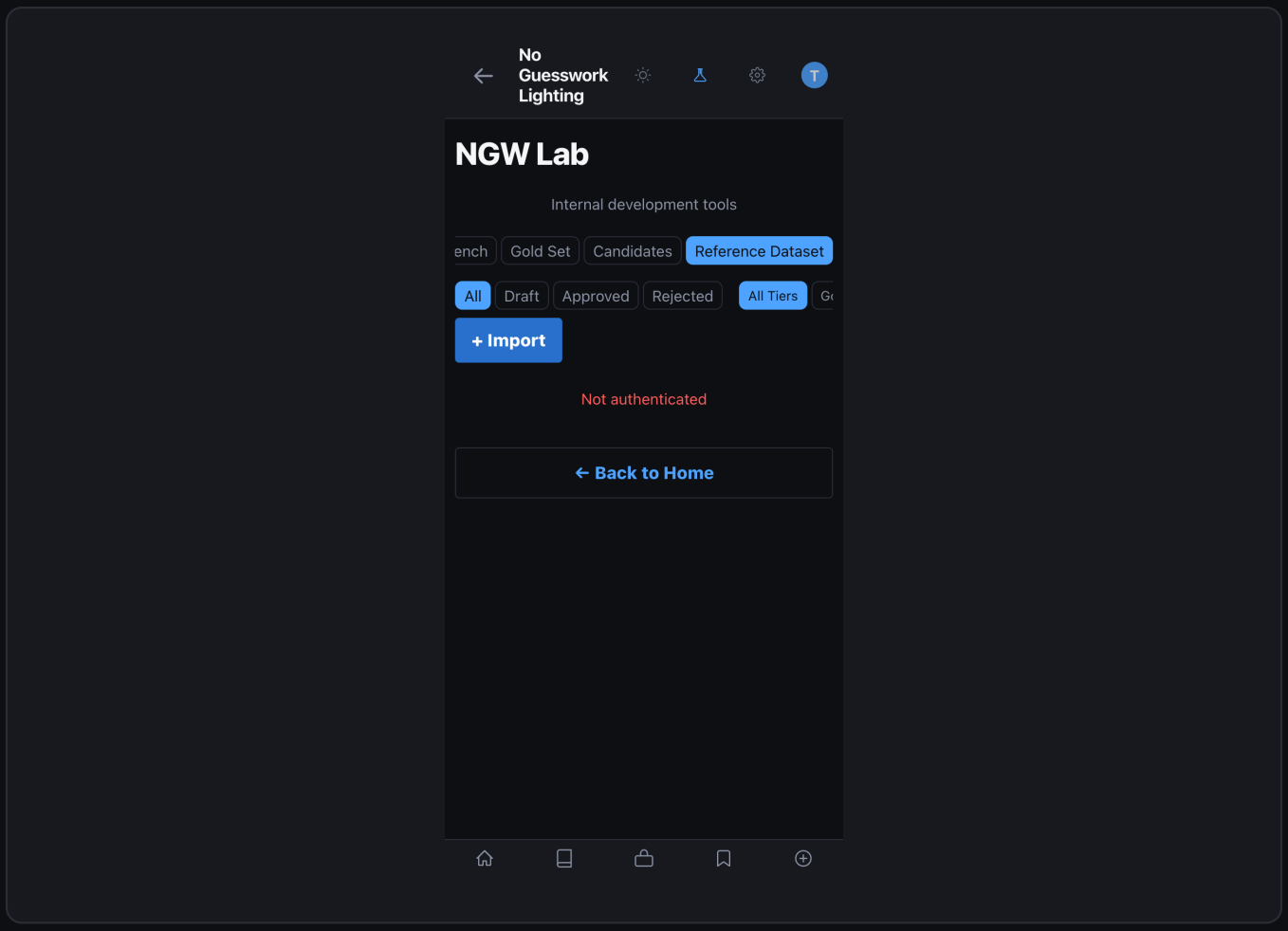
Status	Meaning	Who moves it
proposed	New — awaiting curator review	Creator submits
under_review	Active review in progress	Curator sets
approved	Accepted — ready for engine implementation	Curator approves
deployed	Change live in production engine	Developer deploys

CANDIDATE BEST PRACTICES

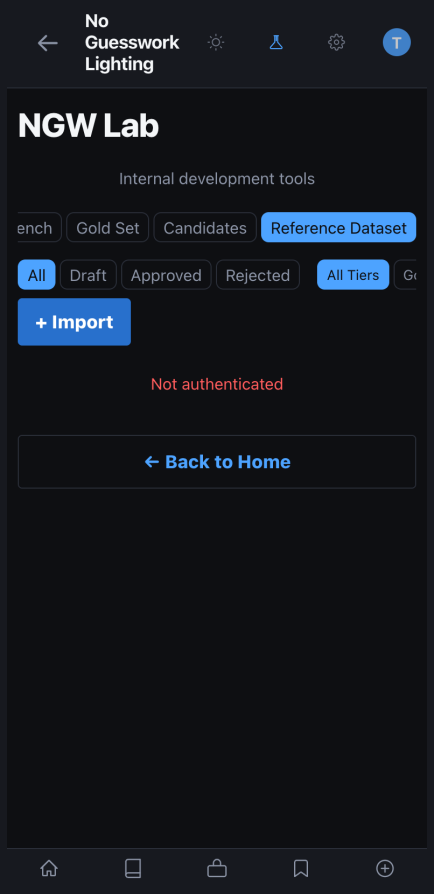
- Every candidate must cite evidence: signal calibration data, case IDs, or gold set IDs.
- proposed_change should be a concrete JSON delta — not vague description.
- Run a full benchmark AFTER implementing an approved candidate before marking deployed.
- If benchmark regresses after implementation: revert candidate status to proposed and add regression note.

Section 4 — Reference Dataset

The reference dataset is the curated library of labeled exemplar images that anchors the reference_read classifier — the highest-priority of the three active pattern-resolution classifiers. The engine loads the dataset at startup and queries it during every analysis. 28 canonical patterns are covered across three quality tiers.



Reference Dataset — grid view with 28-pattern coverage map, tier badges, and approval status



Reference Dataset detail — image with metadata, trust score, and ground_truth structure

28 Canonical Patterns

Family	Patterns
Portrait (studio)	loop, rembrandt, butterfly, clamshell, split, broad, short, paramount
Portrait (natural)	window_portrait, golden_hour, overcast_natural
Fashion / high-key	beauty, high_key_white, glamour, editorial_dramatic
Cinematic / moody	noir, chiaroscuro, low_key_dramatic
Environmental	silhouette, rim_backlit, hair_light_separation

Multi-light	three_point, cross_lighting, kicker_accent
Specialty	catchlight_accent, ambient_fill_dominant, mixed_source

Dataset Entry Metadata Schema

data/reference_dataset/{pattern}/{id}/metadata.json

```
{
  "reference_id":      "loop_gold_001",
  "pattern_id":        "loop",
  "photographer":      "curator@org.com",
  "dataset_tier":      "gold",          # gold | community | synthetic
  "entry_trust_score":  0.90,            # gold default 0.9, community 0.7
  "approval_status":   "approved",       # draft | approved | rejected
  "environment":       "studio",
  "source_type":       "original",
  "light_count":        2,
  "key_direction_deg":  45,
  "shadow_pattern":    "loop",
  "notes":             "Clean catch-light 10 oclock. No ceiling bounce.",
  "ground_truth": {
    "pattern":       "loop",
    "key_angle_deg":  45,
    "key_height":     7.5,
    "fill_ratio":     2.8,
    "modifier_family":"octa"
  },
  "benchmark_metadata": {
    "difficulty":             "medium",
    "category":               "standard",
    "expected_key_direction":  315
  }
}
```

Dataset Tier Reference

Tier	Trust score	Approval	Source
gold	0.90	Dual curator required	Known ground-truth, controlled studio setup
community	0.70	Single curator	User-contributed, curator-verified
synthetic	0.70	None required	Generated by seed_starter_dataset.py

key_direction_deg Convention

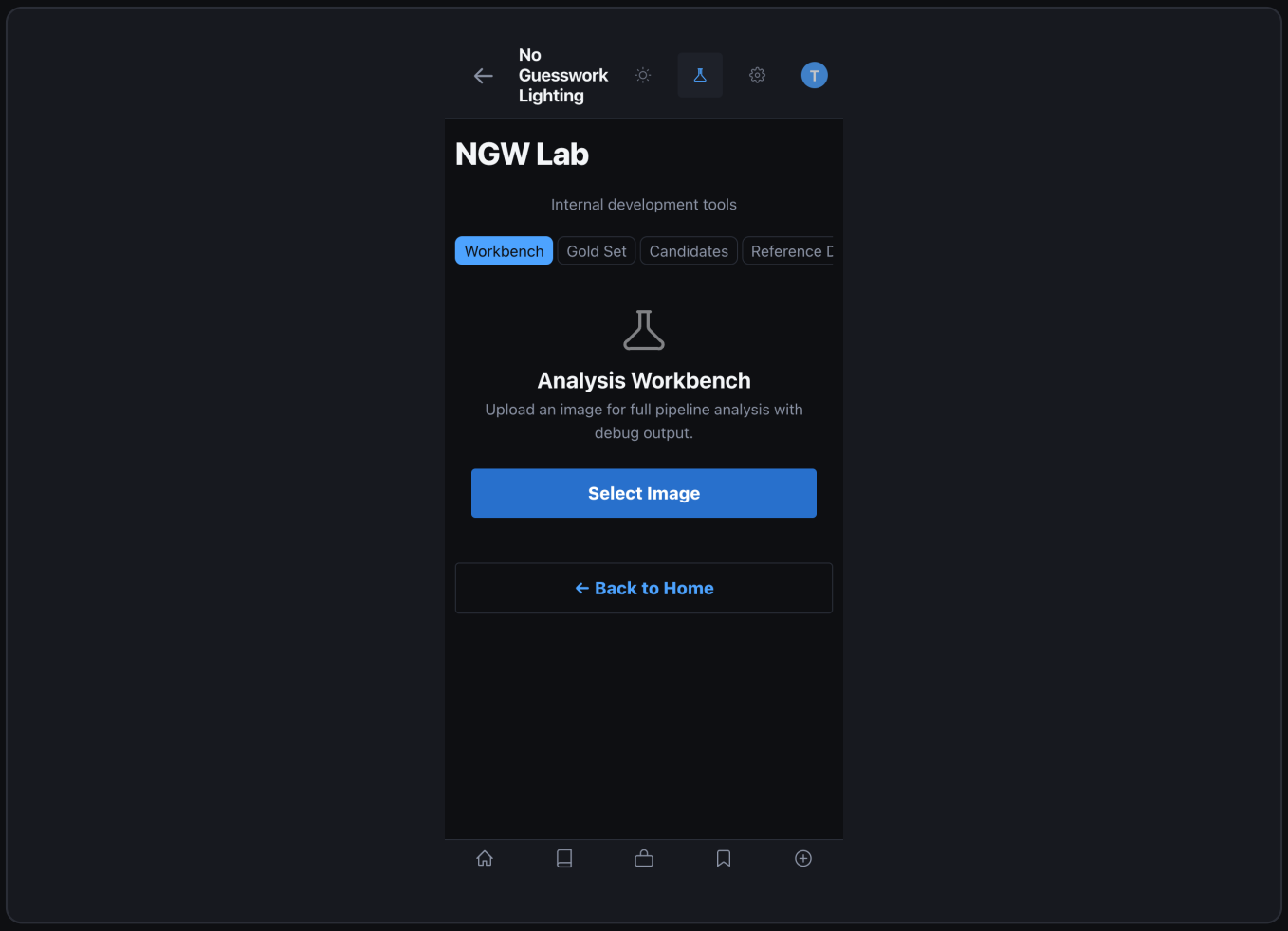
Direction label	key_direction_deg	Typical pattern
upper_left	315 deg	Rembrandt / loop (camera-right side)
upper_right	45 deg	Loop alternate (camera-left side)
left	270 deg	Hard split (camera right)
right	90 deg	Hard split alternate (camera left)
top_center	0 deg	Butterfly / paramount
center / unknown	null	Ambiguous — omit from geo inference

REQUIRED METADATA FIELDS

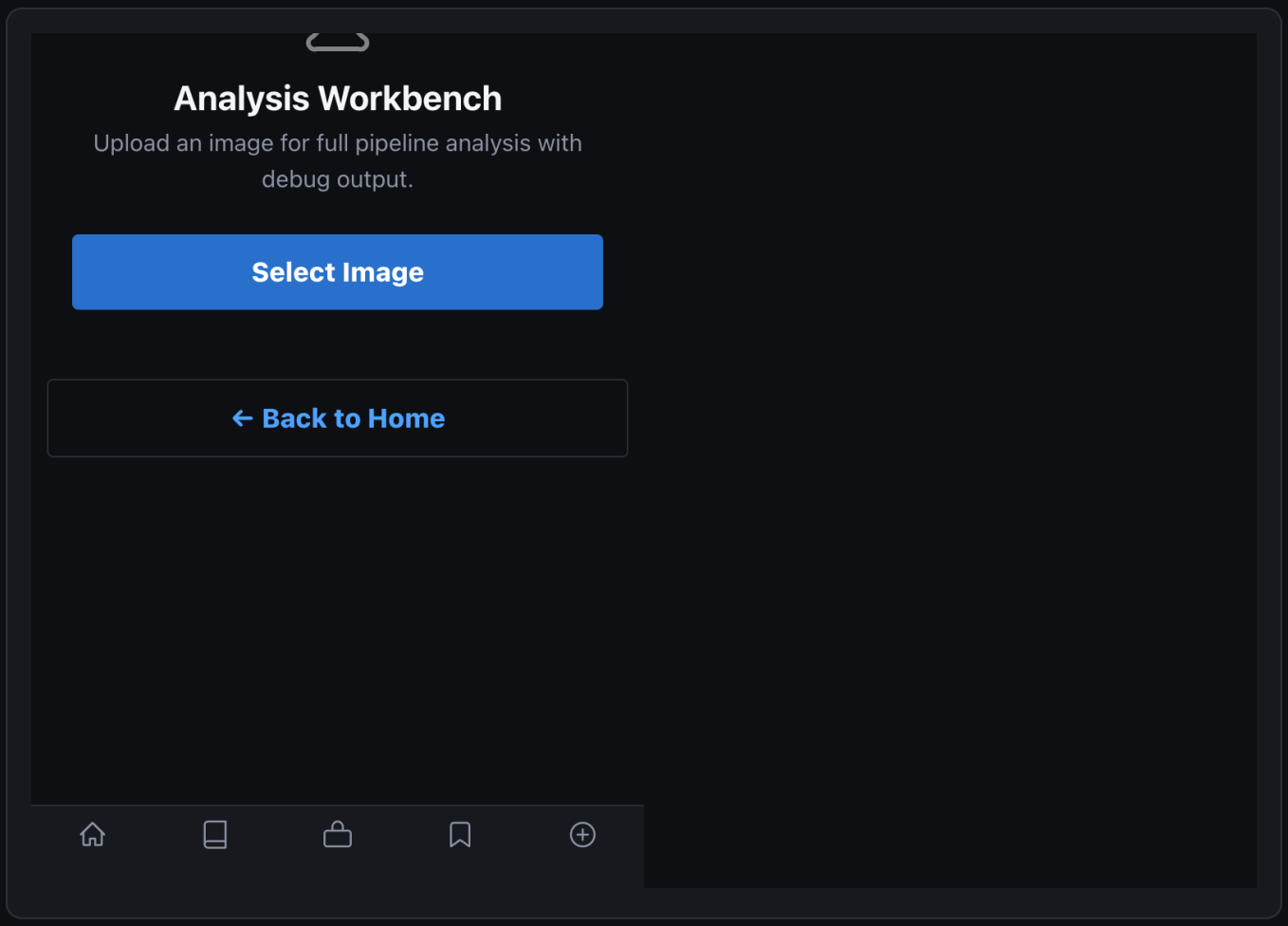
- REQUIRED_META_FIELDS: reference_id, pattern_id, dataset_tier, entry_trust_score.
- verify_dataset.py checks these fields on every entry. Missing = exit code 1.
- VALID_TIERS: gold | community | synthetic.
- VALID_APPROVAL: draft | approved | rejected. Never skip draft before approval.

Section 5 — VLM Analysis Pipeline

Every reference photo analysis runs through a deterministic 7-stage pipeline. The VLM is never asked to name a lighting setup — it extracts observable physical signals only. Pattern determination uses three independent classifiers whose outputs are resolved by the solver chain.



Workbench — debug overlay showing vision pipeline region annotations



Workbench detail — solver chain scores and per-classifier confidence breakdown

7-Stage Pipeline

Stage	Module	What it does
1	engine/vlm.py	VLM call 1 — extract observable signals: shadow, catchlight, geometry, highlights
2	engine/vision_pipeline.py	16+ CV passes — geometric, photometric, and semantic cue extraction

3	engine/cue_inference.py	4-stage inference: geometry source quality environment setup family
4	engine/vlm_reconstruction.py	VLM call 2 — reason about signals, build reconstruction candidates
5	engine/orchestrator.py	Solver chain: consensus consistency contradiction simulator validator
6	engine/orchestrator.py	Pattern resolution: 3 active classifiers ranked candidates
7	engine/reference_read.py	Reference read: cross-validate with dataset authoritative_pattern

Stage 1 — VLM Signal Extraction

The first VLM call uses a system prompt that explicitly restricts the model to observable physical signals only: shadow edges, catchlight shape and position, highlight geometry, colour temperature context. It must never name a setup.

Output	Fields
VLMDescription	subject_type, skin_tones, framing, pose, expression, styling, mood
VLMSignals	geometry, shadows, highlights, catchlights (all physical observables)

Provider selection: VLM_PROVIDER=auto detects openai first, then anthropic. Default models: GPT-4.1 (OpenAI) | claude-sonnet-4-20250514 (Anthropic). Set VLM_PROVIDER=none to disable VLM and fall back to rule-based only.

Stage 2 — Vision Pipeline (16+ Passes)

Pass group	Passes included

Geometry	geometry_pass, pose_solver_pass, inverse_square_solver_pass
Shadow	shadow_pass, shadow_penumbra_pass, occlusion_shadow_pass, multi_shadow_analysis
Light source	highlight_pass, specular_surface_pass, light_direction_field_pass
Catchlight	catchlight_pass (shape, topology, position — all three separate cues)
Environment	background_pass, window_geometry_pass, solar_geometry_pass, environment_light_pass
Modifier	modifier_shape_solver_pass, color_temperature_pass, bounce_geometry_pass
Composite	light_role_support_signals, global_uncertainty_notes

Output: VisualCueReport with 15+ structured cue objects — ShadowEdgeHardness, PrimaryShadowDirection, VerticalLightAngle, CatchlightPosition, CatchlightShape, CatchlightTopology, HighlightGeometry, ReflectionSignatures, BackgroundAnalysis, EnvironmentIndicators, ColorTemperatureContext, LightCountSignals, MultiShadowAnalysis, OcclusionPatterns, CompositeConfidence.

Stage 3 — 4-Stage Cue Inference

Inference stage	Output
GeometryInference	Light direction, height, count, fill ratio, shadow pattern
SourceQualityInference	Modifier family, transition character (soft / hard)
EnvironmentInference	Natural vs studio, background type, special cases
SetupFamilyInference	Primary hypothesis + alternate patterns + ambiguity notes

Stage 4 — VLM Reconstruction (12 Dimensions)

Reconstruction dimension	Description
Dominant source direction	Clock-position estimate from shadow + catchlight geometry
Dominant source height	Vertical elevation inference (low / eye / high)
Source size class	Large-soft / medium / small-hard modifier inference
Source distance class	Near / mid / far placement inference
Modifier family candidates	Octa / umbrella / dish / strip / bare / window (ranked)
Environment	Studio / natural / mixed / outdoor
Light count	Single / two-light / multi-light inference
Light roles	Key / fill / rim / hair / background role assignment
Negative fill likelihood	Probability that fill side uses absorption (black v-flat)
Background lighting likelihood	Probability of a dedicated background light
Bounce likelihood	Probability of bounce fill or ceiling bounce contamination
Ambiguity assessment	Structured uncertainty notes propagated to solver chain

Stage 5 — Solver Chain

Solver	Role
consensus_solver	Aggregate classifier outputs into weighted consensus scores
consistency_engine	Verify cue combinations obey physical lighting constraints
contradiction_engine	Flag and resolve contradictory cue pairs
lighting_simulator	Forward-simulate candidate patterns against the full cue report

hypothesis_validator	Score each hypothesis against all available evidence
solver_trace	Append full reasoning chain to output — never replaces upstream data

Stage 6 — 3-Classifier Pattern Resolution

resolve_pattern_candidates() in engine/orchestrator.py runs three independent classifiers in strict priority order. Higher priority wins on ties.

Priority	Classifier	Method	Evidence used
0 (high)	reference_read	build_lighting_read()	3-layer shadow/highlight/gobo vs dataset
1	catchlight_inference	_infer_pattern_from_catchlights()	Catchlight topology, shape, and count
2	shadow_inference	_infer_shadow_pattern()	Shadow direction + vertical height angle
fallback	rule-based	classify_lighting_pattern()	Mood + modifier + gear (no-image paths only)

AnalysisResult Output Structure

engine/image_analysis_models.py — AnalysisResult


```
{
  "authoritative_pattern": "loop",
  "pattern_candidates":    [{"pattern":"loop","score":0.84},{"pattern":"rembrandt"…
  "confidence_score":      0.84,
  "visual_cue_report":     { /* 15+ VisualCue objects */ },
  "solver_quality": {
    "consistency_score":   0.91,
    "contradiction_count": 0,
    "ambiguity_level":     "low"
  },
  "reference_data": { "matched_entries": [...], "dataset_tier": "gold" },
  "reconstruction": {
    "primary":   { "direction": "upper_left", "height": 7.5, "modifier": "octa" },
    "alternates": [...],
    "uncertainty_notes": []
  }
}
```

Section 6 — Blueprint System

Blueprints are the output specification layer — 19 YAML files in data/systems/canonical/ that map a resolved pattern to exact physical light placement instructions with capture settings, failure modes, and substitution guidance. Loaded at startup, validated by validate_system_yamls.py.

Complete Blueprint YAML Schema

data/systems/canonical/{pattern}.yml (part 1 — header + environment + camera)

```yaml
pattern:      loop
pattern_name: Loop
category:     portrait
difficulty:   medium        # easy | medium | hard
setup_time_minutes: 8
version: "1.0"
status: "active"            # active | experimental | deprecated

environment:
  required: false
  ceiling_height_min_ft: 9.0
  background: seamless
  background_distance_ft: 6.0

subject:
  distance_from_camera_ft: 8.0
  position: center

camera:
  height: 5.5
  lens: 85mm
  angle_to_subject_deg: 0.0

capture_settings:
  iso:           100-400
  aperture:      f/2.8-f/8
  shutter:       1/125-1/250
  white_balance: 5500K
  notes: ["Meter off subject face"]
```

data/systems/canonical/{pattern}.yml (part 2 — lights + shadow + failure + substitutions)

```yaml
lights:              # Array — at least one role:key required
  - role: key
    modifier: octa_90cm
    angle_deg: 45     # horizontal off-axis (0=front, 90=hard side)
    height: 7.5       # feet above floor
    height_offset_in: 0
    distance_ft: 6.0
    power_ratio: 1.0  # relative to key (key always 1.0)
    notes: ["Catch-light should appear at 2 oclock"]
  - role: fill
    modifier: v_flat_white
    angle_deg: 315
    height: 5.5
    distance_ft: 4.0
    power_ratio: 0.33

shadow_signature:
  expected_pattern: loop
  nose_shadow:  "Loop below and to the side of nose"
  cheek_shadow: "Triangle shadow fill side"
  contrast: 0.65    # 0=flat, 1=maximum
  softness: 0.70    # 0=hard, 1=very soft

failure_modes:
  - name: flat_lighting
    description: "Fill too bright, loop shadow disappears"
    recovery:    "Move fill back or reduce fill power by 1 stop"

substitutions:
  - if_missing:  octa_90cm
    use:          umbrella_60cm
    tradeoff:     "Slightly harder transition, shadow stays visible"

use_cases: ["Portrait", "Editorial", "Headshots"]
example_photographers: ["Platon", "Annie Leibovitz"]
```

CLOCK-POSITION CONVENTION

- angle_deg = horizontal off-axis from camera forward. 0 = directly in front.
- 45 deg = upper right of subject (camera right side). Catch-light at ~2 oclock.
- 315 deg = upper left of subject (camera left side). Catch-light at ~10 oclock.
- 90 deg = hard side-light. 270 deg = hard side-light (opposite).
- power_ratio is relative: key = 1.0 always. Fill at 0.33 = 1.6 stop under key.

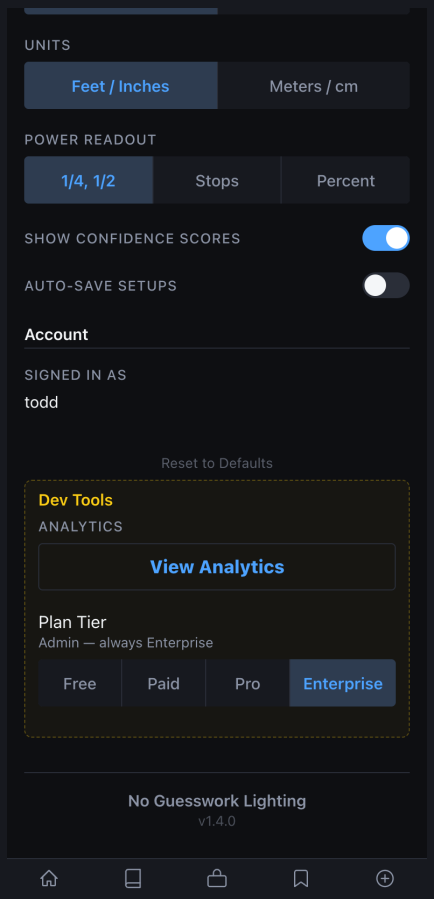
19 Active Blueprint Files

File	Pattern	Difficulty	Light count
loop.yml	Loop	medium	2 (key + fill)
rembrandt.yml	Rembrandt	medium	2 (key + reflector)
butterfly.yml	Butterfly / Paramount	easy	1-2 (frontal key)
clamshell.yml	Clamshell	medium	2 (top key + bottom fill)
split.yml	Split	easy	1 (hard 90 deg)
broad.yml	Broad	easy	2 (key shadow-side visible)
short.yml	Short	medium	2 (key lit-side hidden)

beauty.yml	Beauty	hard	3-4 (dish, fill, kicker)
window_portrait.yml	Window Portrait	easy	1 (natural window)
three_point.yml	Three-Point	medium	3 (key, fill, rim)
editorial_dramatic.yml	Editorial Dramatic	hard	2-3 (hard key + accent)
noir.yml	Noir	hard	1-2 (hard low-key)
high_key_white.yml	High Key White	medium	4+ (key + bg lights)
rim_backlit.yml	Rim / Backlit	medium	2 (rim + fill)
cross_lighting.yml	Cross Lighting	medium	2 (crossed 90 deg)
chiaroscuro.yml	Chiaroscuro	hard	1 (dramatic shadow)
kicker_accent.yml	Kicker Accent	medium	3 (key, fill, kicker)
golden_hour.yml	Golden Hour	easy	1 (natural warm)
ambient_fill_dominant.yml	Ambient Fill	easy	0 (natural ambient)

Section 7 — CLI Tools

All Lab operations can be driven from scripts/. Every script exits 0 on success and 1 on failure, enabling clean CI pipelines.



Dev Tools panel — database status, migration runner, and signal count monitor

verify_dataset.py

Flag	Effect
(none)	Full verification: all 28 patterns, image IO, metadata schema
--quick	Metadata-only — skip image file reads (faster for CI)

--fix	Auto-fix: regenerate thumbnails, rebuild manifest, reset bad fields

bash — verify_dataset.py

```
# Full verification
$ python3 scripts/verify_dataset.py

# Quick metadata check (no image IO)
$ python3 scripts/verify_dataset.py --quick

# Auto-fix minor issues
$ python3 scripts/verify_dataset.py --fix

# Checks: all 28 CANONICAL_PATTERNS present; REQUIRED_META_FIELDS
# present; valid dataset_tier and approval_status values;
# coverage report with missing_high_priority patterns.
# Exit 0 = PASS.  Exit 1 = FAIL (CI-safe).
```

seed_starter_dataset.py

Flag	Effect
(none)	Seed all benchmark entries into data/reference_dataset/
--dry-run	Preview — no disk writes
--force	Overwrite existing entries (use after schema changes)
--filter SUBSTRING	Only process benchmark IDs containing SUBSTRING

bash — seed_starter_dataset.py

```
# Normal seed (first run after init_db)
$ python3 scripts/seed_starter_dataset.py

# Preview without writing
$ python3 scripts/seed_starter_dataset.py --dry-run

# Force-overwrite (after schema change)
$ python3 scripts/seed_starter_dataset.py --force

# Seed only loop pattern entries
$ python3 scripts/seed_starter_dataset.py --filter loop

# Writes: image.jpg + metadata.json per entry,
# 200x200 thumbnails, _version.json, manifest with pattern_coverage.
# Seeded signals: signal_source=seeded, all include_in_*=0.
```

validate_system_yamls.py

bash — validate_system_yamls.py

```
# Validate all 19 canonical blueprint YAMLs
$ python3 scripts/validate_system_yamls.py

# Strict mode — fail on warnings (use before PR merge)
$ python3 scripts/validate_system_yamls.py --strict

# Required fields checked:
#   pattern, pattern_name, category, difficulty, version, status
#   environment.ceiling_height_min_ft
#   lights[] array with at least one role=key entry
#   lights[].angle_deg, height, distance_ft, power_ratio
#   shadow_signature.expected_pattern, contrast, softness
#   failure_modes[] with at least one entry
```

Full Script Reference

Script	Purpose	Key flags
verify_dataset.py	Dataset integrity check	--quick, --fix
seed_starter_dataset.py	Bootstrap synthetic reference data	--dry-run, --force, --filter
validate_system_yamls.py	Lint all 19 blueprint YAML files	--strict
validate_catalog_and_packs.py	Validate gear catalog completeness	--verbose
run_benchmarks.py	Pattern accuracy evaluation (manual)	--pattern=X, --limit=N
nightly_benchmark.py	Full CI benchmark suite (JSON output)	--output=json
ci_benchmark.sh	CI wrapper — exits 1 on regression	--exclude-pattern=X
capture_screenshots.py	Auto-capture UI screenshots for docs	--output-dir=PATH
generate_ngw_docs.py	Build this PDF documentation suite	(none)
generate_lab_manual.py	Build standalone Lab Manual PDF	(none)

Section 8 — Environment Configuration

NGW uses a single .env at the project root. Copy .env.example and fill in values before starting. Never commit .env to version control.

.env Variable Reference

Variable	Required	Description	Default
NGW_JWT_SECRET	Yes	JWT signing secret for Lab API auth	change-me
NGW_DEV_EMAILS	Yes	Comma-separated Lab access allowlist	(none)
NGW_DEV_MODE	No	Bypass JWT auth locally. NEVER in prod.	0
VLM_PROVIDER	No	auto | openai | anthropic | none	auto
VLM_MODEL	No	Override default model name	gpt-4.1
OPENAI_API_KEY	Cond.	Required if VLM_PROVIDER=openai or auto+OpenAI	(none)
ANTHROPIC_API_KEY	Cond.	Required if VLM_PROVIDER=anthropic or auto+Anthropic	(none)
ALLOWED_ORIGINS	No	CORS origins, comma-separated	localhost:5173
LOG_LEVEL	No	DEBUG | INFO | WARNING | ERROR	INFO
HOST	No	Server bind address	0.0.0.0
PORT	No	Server port	8000

VLM_PROVIDER=AUTO BEHAVIOUR

- auto checks OPENAI_API_KEY first, then ANTHROPIC_API_KEY.
- First key found determines which provider is used for the session.
- Set VLM_PROVIDER=none to disable VLM — analysis returns rule-based results only.
- VLM_MODEL override is optional. Defaults: GPT-4.1 (OpenAI) | claude-sonnet-4-20250514 (Anthropic).

Server Startup Sequence

bash — full first-time startup

```
# 1. Copy .env and fill in required values
$ cp .env.example .env

# Minimum required entries in .env:
#   NGW_JWT_SECRET=your-secret-here
#   NGW_DEV_EMAILS=your@email.com
#   OPENAI_API_KEY=sk-...  (or ANTHROPIC_API_KEY)

# 2. Install Python dependencies
$ pip install -r requirements.txt

# 3. Initialise the database
$ python3 -c "from db.database import init_db; init_db()"

# 4. Seed starter data
$ python3 scripts/seed_starter_dataset.py

# 5. Verify dataset integrity
$ python3 scripts/verify_dataset.py

# 6. Validate blueprint YAMLs
$ python3 scripts/validate_system_yamls.py --strict

# 7. Start API server
$ uvicorn api.main:app --reload --port 8000

# 8. Start frontend
$ cd ui && npm install && npm run dev

# 9. Enable Lab in UI
# Settings -> Developer Tools -> toggle on -> enter your NGW_DEV_EMAILS address
```

Section 9 — How-To Workflows

How To: Investigate a Failing Pattern

1. **Check calibration**
   GET /api/lab/signals/calibration?days=30 — find overconfident: true patterns.

2. **Check sample size**
   Ignore patterns with sample_count < 20. Calibration data needs volume.

3. **Run targeted benchmark**
   POST /api/lab/benchmarks/run with notes describing which pattern to watch.

4. **Review worst cases**
   GET /api/lab/benchmarks/runs/{id}/results — sorted worst-first by final_score.

5. **Inspect case history**
   GET /api/lab/benchmarks/cases/{id}/history — see if regression is new or old.

6. **Check blueprint YAML**
   Open data/systems/canonical/{pattern}.yml — verify angle_deg, shadow_signature.

7 **Validate YAML**
python3 scripts/validate_system_yamls.py --strict

8 **Check dataset coverage**
python3 scripts/verify_dataset.py — confirm pattern has approved gold entries.

9 **Create rule candidate**
POST /api/lab/rule_candidates with title, description, rationale, proposed_change.

10 **Re-benchmark after fix**
POST /api/lab/benchmarks/run to confirm improvement. Promote baseline if clean.

How To: Add a Gold Set Entry

1 **Choose a clean original**
Unedited JPEG with visible lighting geometry. Minimum 1024px wide.

2 **Confirm ground truth**
Record actual setup: key angle, height, modifier, fill side, ratio.

3 **Copy to dataset path**
data/reference_dataset/{pattern}/{id}/image.jpg

4 **Write metadata.json**
Fill all REQUIRED_META_FIELDS: reference_id, pattern_id, dataset_tier, entry_trust_score.

5 **Set dataset_tier correctly**
gold = 0.90 trust, dual curator. community = 0.70, single curator.

6 **Start as draft**
approval_status: draft always. Never commit approved without review.

7 **Run verify_dataset.py**
python3 scripts/verify_dataset.py — confirms entry passes schema check.

8 **Get second curator review**
Gold tier requires two curator approvals before approval_status: approved.

9 **Promote to benchmark case**
POST /api/lab/benchmarks/cases/from-gold-set/{id} to auto-infer case fields.

How To: Set Up CI Benchmark Gate

1 **Seed and verify locally**
Run seed + verify_dataset.py + validate_system_yamls.py all passing.

2 **Run first benchmark**
POST /api/lab/benchmarks/run — establish initial scores.

3 **Promote baseline**
POST /api/lab/benchmarks/baseline — pin the run as reference.

4 **Add CI step**
Call POST /api/lab/benchmarks/ci-run with X-CI-Secret header.

5 **Check exit_code**
exit_code: 0 = all gates passed. exit_code: 1 = deploy blocked.

6 **Configure thresholds**
Gate: overall < 0.70 OR any pattern < 75% OR regression > 5pp = fail.

7 **Update baseline post-fix**
After any approved candidate is implemented: re-run, then POST baseline.

Section 10 — Troubleshooting

Symptom	Likely cause	Fix
401 on all /api/lab/* endpoints	Missing or expired JWT	Re-auth; verify NGW_JWT_SECRET matches
Lab tab not visible in UI	Email not in NGW_DEV_EMAILS	Add email to .env, restart server
POST /api/lab/signals returns 422	Missing required pattern_id	Include pattern_id in request body
Signals all show include_in_*=0	signal_source=seeded or internal	Only live source sets include_in_*=1 by default
Benchmark run shows 0 cases evaluated	No benchmark_cases rows in DB	POST cases or use from-gold-set promotion
CI exit_code=1 with no regression_flag	overall_score below 0.70 gate	Review worst-scoring cases; tune blueprint
VLM provider not called	No API key found by auto-detect	Set OPENAI_API_KEY or ANTHROPIC_API_KEY
validate_system_yamls fails	Missing required field in blueprint YAML	Check lights[] array and shadow_signature fields
verify_dataset.py exits 1	Pattern missing entries or bad metadata	Run --fix or manually correct metadata.json
seed_starter_dataset errors	Entries already exist	Run with --force to overwrite
reference_read returns null pattern	No approved entries for that pattern	Add entry with approval_status: approved

Candidate stuck in under_review	No curator has advanced the status	PUT /api/lab/rule_candidates/{id} status: approved

Diagnostic Sequence

1. **Health check**
   GET /health — should return { "status": "ok" }.

2. **Lab auth check**
   GET /api/lab/signals/hygiene — 401 = JWT issue; 200 = Lab auth working.

3. **Dataset check**
   python3 scripts/verify_dataset.py --quick — fast metadata-only pass.

4. **Blueprint check**
   python3 scripts/validate_system_yamls.py — find any YAML schema errors.

5. **Signal hygiene**
   GET /api/lab/signals/hygiene — verify live vs seeded counts are expected.

6. **Baseline check**
   GET /api/lab/benchmarks/baseline — confirm has_baseline: true.

7 **Quick benchmark**
POST /api/lab/benchmarks/run with case_limit=5 — fast pass/fail check.

8 **Inspect worst case**
GET /api/lab/benchmarks/runs/{id}/results — read the error_msg field.

Appendix — Complete API Reference

All /api/lab/* endpoints require Authorization: Bearer with a JWT whose email is listed in NGW_DEV_EMAILS. Exception: POST /api/lab/signals is public — no auth required.

Method + Endpoint	Auth	Description
POST /api/lab/signals	Public	Record session outcome signal
GET /api/lab/signals/summary	Dev JWT	Headline KPIs — days + source params
GET /api/lab/signals/patterns	Dev JWT	Per-pattern aggregated metrics
GET /api/lab/signals/calibration	Dev JWT	Confidence vs outcome gap per pattern
GET /api/lab/signals/recent	Dev JWT	Latest N signals (default source=live)
GET /api/lab/signals/hygiene	Dev JWT	Source breakdown and eligibility flags
POST /api/lab/signals/seed	Dev JWT	Insert 45 synthetic bootstrap rows
POST /api/lab/benchmarks/cases	Dev JWT	Create a benchmark test case
GET /api/lab/benchmarks/cases	Dev JWT	List cases (?pattern_id=X&difficulty;=hard)
PUT /api/lab/benchmarks/cases/{id}	Dev JWT	Update case fields or notes
DELETE /api/lab/benchmarks/cases/{id}	Dev JWT	Remove a benchmark case

GET /api/lab/benchmarks/cases/{id}/history	Dev JWT	Score history for one case across runs
POST /api/lab/benchmarks/cases/from-gold-set/{id}	Dev JWT	Promote gold_set_entries row to case
POST /api/lab/benchmarks/run	Dev JWT	Trigger a manual benchmark run
POST /api/lab/benchmarks/ci-run	CI Secret	CI-triggered run — returns exit_code
GET /api/lab/benchmarks/runs	Dev JWT	List benchmark runs (?limit=N)
GET /api/lab/benchmarks/runs/{id}/results	Dev JWT	Per-case results sorted worst-first
GET /api/lab/benchmarks/pattern-metrics	Dev JWT	Per-pattern metrics with delta vs last run
GET /api/lab/benchmarks/baseline	Dev JWT	Current baseline run info
POST /api/lab/benchmarks/baseline	Dev JWT	Promote latest completed run to baseline

DATA BACKUP

- All lab data is in the SQLite DB at data/ngw.db (dev) or Postgres (prod).
- Reference dataset images are in data/reference_dataset/ — back up alongside DB.
- Before any schema migration: sqlite3 data/ngw.db .dump > backup_$(date +%Y%m%d).sql
- _version.json in reference_dataset/ tracks schema_version — check after updates.